Polynomial stability and computability

Proof explanations, formal interfaces, and verified scope

Companion to the Lean development of Jun Liu and Ivan O. Shevchenko

31 October 2026

1 Scope and preservation of the published proof

This companion describes new work in the `Stability` library. The reference points are the published Lean commit `81276ab` and the revised manuscript [1] at Overleaf commit `6f5bced`, in `new Main Document`. The published `HGAS` library, its public theorem statements, and its Lean/Mathlib 4.30.0 environment are preserved. The two earlier companions continue to document their respective releases.

The new library is not a claim that every revised intermediate statement has already been formalized. The bounded-degree polynomial GAS and LAS corollaries and uniform dimension-and-odd-degree-input completeness theorem below are proved. The Fourier estimate, recurrence lemma, restart-budget lemma, and cubic-relaxation estimates are completed supporting results, with the limits of their correspondence stated explicitly. The broader source-level trajectory/test equivalence and the Section 9 compact-base trajectory extraction are now proved as well. The canonical-graph size and polynomial Fourier bounds are explicit. The full halting-to-HLS reduction and its cubic/cofinal hardness consequences are now proved. The printed gain and reward have their exact two-sided invariant-average bounds: the positive branch constructs a probability on the actual compact graph, and the negative branch uses ordinary-time extraction. The printed-gain compiler is computable and gives the fixed-pair and cofinal GAS completeness theorems. The existing alternative-gain compiler remains unchanged. `FORMALIZATION_LEDGER.md` records the current status.

The simulation estimates follow the written proof in [2]; the author's earlier Lean implementation is linked in the README. No code is copied or ported from it.[1]

2 Local attraction becomes global under homogeneity

The formal definition of a forward trajectory includes its initial value and the differential equation on the entire nonnegative time ray. Merely quantifying over such trajectories can be vacuous when a solution blows up in finite time. We therefore include local existence explicitly.

Definition 2.1. *An equilibrium x_* is locally asymptotically stable if it is Lyapunov stable with local existence of global forward trajectories, and there is $r > 0$ such that every initial state in $B_r(x_*)$ admits a global forward trajectory and every such trajectory tends to x_*.*

Here stability with existence means that for every $\varepsilon > 0$ there is $\delta > 0$ such that every initial state in $B_\delta(x_*)$ admits a global forward trajectory, and every such trajectory stays in $B_\varepsilon(x_*)$. This

[1] Parts of that implementation were inspected during an earlier feasibility review; no clean-room claim is made.

is the nonvacuous convention already used by the published HLS development. For locally unique polynomial ODEs it agrees with the usual initial-value formulation. The theorem below is proved for the stated trajectory predicates and does not assume a uniqueness theorem.

Theorem 2.2. *Let E be a real normed vector space, let $k \in \mathbb{N}$, and suppose*

$$F(cx) = c^{k+1}F(x) \qquad (c \in \mathbb{R},\ x \in E).$$

Then the origin is locally asymptotically stable if and only if it is globally asymptotically stable.

Proof. Global stability and attraction imply their local counterparts, so consider the other direction. The published homogeneous stability lemma gives a global forward trajectory from every initial state: shrink the state into a neighborhood where existence is known, then scale its trajectory back. The same lemma gives the global linear trajectory bound equivalent to Lyapunov stability with existence. For the present theorem we need its existence conclusion; the local stability hypothesis already gives stability.

It remains to prove attraction from an arbitrary initial state x_0. Let $r > 0$ be a local attraction radius, and put

$$c = \frac{r}{2(\|x_0\| + 1)} > 0.$$

Then $\|cx_0\| < r$. If γ is any forward trajectory from x_0, homogeneity gives a forward trajectory

$$\widetilde{\gamma}(s) = c\gamma(c^k s)$$

from cx_0. Given $\varepsilon > 0$, local attraction gives $T \geq 0$ with $\|\widetilde{\gamma}(s)\| < c\varepsilon$ for all $s \geq T$. For $t \geq Tc^k$, set $s = t/c^k$. Then

$$c\|\gamma(t)\| = \|\widetilde{\gamma}(s)\| < c\varepsilon.$$

Dividing by $c > 0$ proves attraction. The choice of c also handles $x_0 = 0$, so no separate zero-state argument is needed. □

The definitions and proof are in `Stability/Dynamics/LocalAsymptoticStability.lean`. The main declaration is `locallyAsymptoticallyStable_iff_globallyAsymptoticallyStable`. It reuses the published trajectory-scaling theorem and homogeneous linear-bound characterization [2, Lemma 1.5].

3 General polynomial GAS and LAS

The input is a finite sparse rational code for a polynomial vector field. Our new languages `PGASQnd` and `PLASQnd` require each decoded coordinate polynomial to have total degree at most d, require the origin to be an equilibrium, and require GAS or LAS, respectively. The degree bound is imposed *after decoding*: two entries of arbitrarily high degree may cancel. A bound on the largest syntactic exponent would describe a different input condition.

Theorem 3.1. *Let N_* be the fixed dimension threshold of the published cubic HGAS completeness theorem. For every fixed $n \geq N_*$ and $d \geq 3$, both `PGASQnd` and `PLASQnd` are Σ_1^0-hard. Each is undecidable, and its complement is not recursively enumerable.*

Proof. Homogeneous cubic GAS is Σ_1^0-complete in dimension n by the published theorem. A reduction must nevertheless handle every raw input, including codes that are not homogeneous cubic fields. Define

$$C(F) = \begin{cases} F, & \text{if every decoded coordinate of } F \text{ is homogeneous of degree 3,} \\ 0, & \text{otherwise.} \end{cases}$$

The homogeneity test is primitive recursive, so this is a total primitive- recursive map. For a valid input, its decoded degree is at most three and the GAS assertion is unchanged. For an invalid input, $C(F)$ is the zero field. In positive dimension this field is not GAS: a nonzero constant trajectory does not tend to the origin. The existing threshold N_* is positive, so this case applies throughout the theorem.

For LAS, Theorem 2.2 identifies LAS and GAS on every valid homogeneous cubic input. It also excludes LAS of the zero fallback, which is homogeneous of every positive degree. Thus the same map C is a many-one reduction to both target languages. No parity restriction on d is needed, because d is an upper bound rather than a specified homogeneous degree. Undecidability and failure of recursive enumerability of the complements follow by pulling a hypothetical decision or complement semidecision procedure back along the reduction. □

These are hardness results, not completeness results: this argument does not show that general polynomial GAS or LAS is recursively enumerable. The two modules are

`Stability/Computability/PolynomialGAS.lean`
`Stability/Computability/PolynomialLAS.lean`.

Their correctness declarations include both branches of the raw-code reduction:

`cubicToPolynomialCode_correct`
`cubicToPolynomialCode_correct_las`.

The endpoint names begin with `pgasQ_` and `plasQ_`.

4 A single checker when the degree is input data

For fixed dimension n, the language `VariableOddHGASQn` takes a pair (d, F) as input and requires d to be odd and F to belong to `HGASQnd`. One must not infer its recursive enumerability merely from the existence of a semidecision procedure for each fixed d.

Instead, `Stability/Computability/VariableHGASDegree.lean` constructs one primitive-recursive Boolean relation on (d, F, c), where $c \in \mathbb{N}$ encodes a rational Lyapunov candidate. It checks parity and homogeneity, then runs the published exact-grid positivity tests on the candidate and its derivative numerator. The candidate type and its decoder depend on n, but not on d. The published soundness theorem and rational homogeneous converse show that an accepted witness exists exactly for inputs in the semantic language. Searching the natural witnesses therefore proves `variableOddHGASQ_re` uniformly in the degree input.

The computable map $F \mapsto (3, F)$ embeds the fixed cubic language. Thus `variableOddHGASQ_sigmaOneComplete` proves completeness for every fixed $n \geq N_*$. This is a degree-input result with an explicit oddness condition.

4.1 A uniform checker with dimension as input

`VariableDimensionHGAS.lean` now supplies the further uniformity. An input is (n, d, F), where F is a list of sparse polynomials and each exponent vector is itself a list of natural numbers. The language `DimensionDegreeHGASQ` requires exactly n field coordinates, exactly n entries in each exponent vector, odd d, and the existing semantic predicate `HGASQnd` for the decoded field. Padding makes decoding total, but malformed shapes are rejected before this can affect accepted semantics. Coefficients of duplicate monomials are added exactly, so cancelling terms do not create false degree restrictions.

A witness has a dimension-independent type: a natural exponent, one raw polynomial, and two natural grid resolutions. The raw arithmetic modules implement derivatives, the rational Lyapunov derivative numerator, and the grid tests directly with list operations. Cartesian grid words are generated by recursion on the input dimension. These operations form one jointly primitive-recursive Boolean checker, proved to commute exactly with the old fixed-dimensional checker on well-formed codes. An unsuccessful natural-number witness decode produces zero grid resolutions, which are rejected even in dimension zero.

Write $\mathcal{H}$ for `DimensionDegreeHGASQ` and C for `RawHGASChecker.check`. The published soundness and converse theorems give

$$(n, d, F) \in \mathcal{H} \quad \Longleftrightarrow \quad \exists c \in \mathbb{N} : \; C((n, d, F), c) = \text{true}.$$

This proves uniform recursive enumerability. The computable encoding of one fixed cubic slice gives Σ_1^0-completeness, undecidability, and failure of recursive enumerability of the complement. This language deliberately rejects even degree tags; no classification of those tags is inferred from the odd-degree converse theorem. The regression tests include dimension zero, malformed shapes, failed-resolution witnesses, and an accepted certificate for $-x^3$ displayed with cancelling quadratic terms.

These results give partial coverage of Corollary `cor:variable` in the manuscript. Separate dimension-input languages with fixed degree three or fixed odd degree are not formalized. No literal identification with the paper's unrestricted all-degree language is asserted: the joint Lean language admits dimension zero and explicitly requires odd degree.

5 Shifted theta exponents and weighted summability

Step 1 of the revised manuscript's Fourier bound uses the shifted exponent

$$\sigma(n) = (n+1)^2 - \tfrac{1}{4}.$$

It is negative at $n = -1$. Treating every exponent as nonnegative would lose that term. A Gaussian estimate with arbitrary linear and constant parts handles it directly, without a special exception.

Lemma 5.1. *Suppose $a > 0$, $\lambda > 0$, $\rho \geq 1$, $A \geq 0$, and $\lambda\rho^A < 1$. For arbitrary $b, c, B, C \in \mathbb{R}$ and $d \in \mathbb{N}$, the series*

$$\sum_{n\in\mathbb{Z}} (1 + |n|^d)\lambda^{an^2+bn+c}\rho^{A|an^2+bn+c|+B|n|+C}$$

converges.

Proof. Take logarithms of the two exponential factors. Since $\log\rho \geq 0$,

$$|an^2 + bn + c| \leq an^2 + |b||n| + |c|$$

gives an upper bound of the form

$$-\alpha n^2 + \beta|n| + \gamma, \qquad \alpha = -a(\log\lambda + A\log\rho) > 0.$$

The signed linear term $(\log\lambda)bn$ is at most $|b\log\lambda||n|$. Thus the summand is bounded by a fixed polynomial factor times a Gaussian with an arbitrary linear shift. The existing Gaussian summability theorem proves convergence of this majorant. Every inequality holds for every integer n, including those for which $an^2 + bn + c < 0$. □

For $\lambda > 0$, $\rho \geq 1$, and $D \in \mathbb{R}$, take $a = 1$, $b = 2$, $c = 3/4$, $A = 4$, $B = 4D$, $C = 2D$, and $d = 1$. The lemma and $4(2|n| + 1) \leq 8(1 + |n|)$ prove convergence of

$$\sum_{n\in\mathbb{Z}} 4(2|n| + 1)\lambda^{(n+1)^2-1/4}\rho^{4|(n+1)^2-1/4|+2(2|n|+1)D}$$

whenever $\lambda\rho^4 < 1$. This is an all-index majorant corresponding to the revised coefficient and frequency bounds; the printed proof instead separates the finitely many negative exponents. A comparison theorem permits any complex coefficients and frequencies satisfying the corresponding bounds.

These results are in `Stability/Analysis/ShiftedThetaBounds.lean`. The actual graph correspondence additionally requires collecting equal frequencies, finite products, and reciprocal expansions. The later graph and Fourier modules prove those identities and give the explicit coordinate budgets; the scalar majorant alone is not a substitute for that correspondence.

6 Why the intended torus trajectories are recurrent

Lemma 6.1. *Let G be a compact first-countable topological abelian group and let $f : \mathbb{R} \to G$ be an additive homomorphism. For every $x \in G$, the curve $t \mapsto x + f(t)$ returns to its initial point along times tending to $+\infty$. So does every continuous observation of this curve.*

Proof. Compactness gives a strictly increasing sequence $\phi : \mathbb{N} \to \mathbb{N}$ such that $f(\phi(n))$ converges to some $y \in G$. Set $\tau_n = \phi(2n) - \phi(n)$. Strict increase implies $\tau_n \geq n$, so $\tau_n \to +\infty$. Additivity and continuity of subtraction give

$$f(\tau_n) = f(\phi(2n)) - f(\phi(n)) \longrightarrow y - y = 0.$$

Hence $x+f(\tau_n) \to x$. Applying a continuous observation preserves this convergence. No nonresonance assumption is needed. □

The generic result and its finite-torus specialization are in

`Stability/Dynamics/CompactTranslationRecurrence.lean`.

This proves the recurrence step in the revised intended-trajectory argument. It does not assert the full source ODE, validity residuals, or test averages for that graph; those must be connected separately.

7 Forward-only ingredients for the broader selector

The revised selector assumes forward recurrence and bounded forward averages of a real power $\mu \geq 1$. Two points require care when reusing the published formalization: a negative clock must not introduce a reverse-recurrence assumption, and a bound at exponent one cannot be replaced by a bound at exponent two.

7.1 Changing the sign of a clock without reversing time

The unary modular vector field has rational coefficients. Complex conjugation therefore takes a trajectory with clock $i\Omega$ to one with clock $-i\Omega$. It preserves every coordinate magnitude and the same forward return-time sequence. If $\Omega < 0$, conjugate first; then rescale time by the positive factor $\pi/|\Omega|$. The resulting clock is the standard $i\pi$, with forward recurrence intact.

Conjugation commutes with each finite-horizon Cesàro mean. It also changes the first-harmonic factor $e^{-i\alpha}$ to $e^{i\alpha}$, so the corresponding phase is $-\alpha$ and real target moments are unchanged. Positive time rescaling preserves the full forward limits and transports norm-power averages by rescaling their horizons.

These are exact statements in `Stability/Selection/UnaryClockConjugation.lean`. The linked Jacobi module `LinkedClockConjugation.lean` conjugates all residuals and both clocks. An important qualification is that a real elliptic clock is unchanged under conjugation, while a purely imaginary modular clock changes sign. We do not identify this transformation with simultaneous negation of all physical Jacobi frequencies.

7.2 The critical linear cusp estimate

In a cusp, Haar measure has a positive height marginal bounded below by a multiple of dy/y^2. The first height moment already diverges:

$$\int_A^\infty y\frac{dy}{y^2} = \int_A^\infty \frac{dy}{y} = \infty.$$

The module `Selection/LinearCuspDivergence.lean` proves this with the published cusp measure and arbitrary finite cutoffs. Taking the square root of the published nonnegative square-theta observable gives a continuous linear observable on the modular frame quotient. Its cusp lower bound implies infinite Haar integral.

The existing unconditional pointwise horocycle theorem applies in either direction of nonzero speed. On a nonperiodic orbit, continuous compactly supported minorants with arbitrarily large Haar integrals force the linear observable's forward averages to diverge. Along the reconstructed modular trajectory, that observable is exactly the norm of the C coordinate. For every $u \geq 0$ and real $\mu \geq 1$, the elementary inequality $u \leq 1 + u^\mu$ turns an upper bound on the μ-power averages into an upper bound on linear averages. Hence such a trajectory cannot have a nonperiodic frame orbit.

This exclusion is formalized in `Stability/Selection/LinearThetaCuspExclusion.lean`. The endpoint assumes the normalized unary ODE, nonvanishing B, the lambda-coordinate identity, and an actual reconstructed frame path. It proves the nonperiodic-orbit exclusion. The next modules supply the lift and frame from the recurrent trajectory itself.

7.3 Selecting the unary trajectory from forward means

Three modules combine the preceding ingredients:

`OrdinaryTimeGenericity.lean`
`OrdinaryTimeClosedFrame.lean`
`OrdinaryTimeUnarySelection.lean`.

The strongest endpoint starts with an all-real-time linked trajectory whose modular clock is $i\pi$, one forward recurrence sequence of the literal linked polynomial state, all ten linked residuals, and the limits

$$\langle Y \rangle = 1, \qquad \langle p_4 \rangle = 1, \qquad \langle Y(t)\eta e^{-i\pi t} \rangle = \frac{1}{128}, \qquad |\eta| = 1.$$

Here brackets mean the full limit of $T^{-1}\int_0^T$ as real $T \to \infty$. The remaining hypothesis is an eventual upper bound on the corresponding averages of the full linked-state norm to a real power $\mu \geq 1$.

The nonzero harmonic mean excludes the three degenerate divisors. Indeed, recurrence and the unary ODE force Y to be constant on either of the divisors $A = 0$ and $C = 0$. On $B = 0$, the residual $Y^4 = B$ gives $Y = 0$. A constant has zero mean against the nonzero-frequency character, contrary to the stipulated limit. No auxiliary p_2 moment is needed.

On the generic locus, the published reconstruction theorem produces a differentiable lambda lift and either a horizontal path or an exact frame path. The full-state norm controls the unary C coordinate, so the linear cusp argument excludes the nonperiodic frame case. A closed chart follows. Its cusp-amplitude classification and the two raw means $\langle Y \rangle$ and $\langle p_4 \rangle$ select the infinity cusp and unit speed. Finally, the first harmonic is

$$2\eta e^{-\pi y} e^{i\pi x} = \frac{1}{128}.$$

Taking absolute values fixes the gauge height $y = y_*$ with $e^{-\pi y_*} = 1/256$; the same identity then gives $\eta e^{i\pi x} = 1$.

Thus the entire unary state is the canonical theta state at $x+t+iy_*$, and $p_4(t) = \theta_4(x+t+iy_*)^2$. The frame and genericity conditions are conclusions, not assumptions. This linked result still uses the p_4 mean and all linked residuals; it is not a claim to have formalized the standalone unary proposition with only two means.

The module `OrdinaryTimePhysicalClock.lean` handles a general clock $i\Omega$, $\Omega \neq 0$, with recurrence at time zero. For $\Omega > 0$, the substitution $t = (\pi/\Omega)s$ preserves forward horizons and gives the normalized theorem. For $\Omega < 0$, first conjugate, apply the positive-clock theorem, and use the series identity

$$\overline{\theta_j(\tau)} = \theta_j(-\overline{\tau})$$

to return to the original state. The conclusion has the signed speed Ω/π and the same literal phase equation. No step samples the original curve at negative times. The elliptic clock is conjugated and positively rescaled according to its own value; a real elliptic clock is not negated by conjugation.

7.4 The gauged Jacobi output

The linked residuals recover p_3 and the Eisenstein coordinate from the canonical unary state. They recover p_2 up to sign; translating the modulus by two absorbs that sign while preserving its height and the other roots. This supplies a canonical initial root state. The published theta parametrization and reciprocal-chart continuation theorem then produce one globally regular affine Bloch path. The linked forward return sequence projects to the Jacobi coordinates, so the published recurrence criterion forces the Bloch parameter to be real. These steps are combined in `OrdinaryTimeBlochClassification.lean` without assuming a Bloch representation at the outset.

For physical clocks $(\Omega + N)/2$ and $i\Omega$, write $\rho = N/\Omega$ and assume $\Omega \neq 0$ and ρ irrational. The gauge ODE

$$g' = \frac{i(\Omega + 2N)}{4}\, g$$

determines the actual gauge as its initial value times an exponential; no normalization of $g(0)$ is imposed. The gauged output $Z = gT_1$ has the absolutely summable quadratic Fourier representation of that Bloch path.

Let $a_n = R^{n^2} S^n$, with $R, S \neq 0$, be a target coefficient family, and let $c_+, c_- \neq 0$ be the initial character multipliers. The module `OrdinaryTimeJacobiMoments.lean` uses the three further forward

means

$$\langle Z\rangle = 1, \qquad \langle c_+ e^{-i(\Omega+N)t} Z\rangle = c_+ a_1, \qquad \langle c_- e^{-i(\Omega-N)t} Z\rangle = c_- a_{-1}.$$

The published three-moment rigidity theorem identifies the entire actual output with the quadratic trace of the target coefficients. The proof derives the representation, its summability, and the gauge branch from the trajectory data. It does not assume a selected invariant measure or backward recurrence.

7.5 The literal source coordinates and arithmetic detector

The modules from `OrdinaryTimeDetectorProjection.lean` through `OrdinaryTimeDetectorAverage.lean` apply this classification to the actual rational polynomial source field. Pointwise validity identifies its linked residuals, reciprocal variables, and unit clocks. Two extra residuals in the compiler follow algebraically from the eight displayed relations; they do not impose a stronger source condition.

The controller equations and forward recurrence force constant clock parameters. For distinct primes p_j, these parameters give signed frequencies $\pm\sqrt{p_j}$. Integer independence survives both those signs and the common scaling factor. The nonzero-column condition on the integer input then gives the irrational ratios needed above. Thus these ratios, the phase trajectory, and the target theta series are conclusions, not additional assumptions on the source trajectory.

Let Φ denote the continuous arithmetic Fourier detector on the phase torus. The actual polynomial detector $D(x(t))$ satisfies

$$D(x(t)) = c\,\Phi(\varphi_0 + t\omega), \qquad |c| = 1.$$

Here the reciprocal graph equations supply the Poisson factors, and the actual right-hand-side phase ODE supplies the constant c. Absolute Fourier summability and integer independence give the full forward limit

$$\lim_{H\to\infty} \frac{1}{H}\int_0^H D(x(t))\,dt = c\int \Phi.$$

The published arithmetic constant-term theorem says that $\int \Phi = 0$ exactly when the integer system has no solution. Since $c \neq 0$, both real scalar detector means vanish exactly in that case.

This theorem concerns a *given qualifying trajectory*. Its hypotheses are the literal ODE for all real times, pointwise validity, forward recurrence at zero, the specified scalar moment means, and an eventual bound on the full Euclidean μ-moment for some real $\mu \geq 1$. No backward recurrence or selected invariant measure is assumed. Pointwise validity is not inferred from zero residual averages. The compact-base extraction described below supplies these hypotheses from the measure assumptions in Section 9.

7.6 Constructing a trajectory for the converse

The modules from `OrdinaryTimeIntendedOrbit.lean` through `OrdinaryTimePrimeExistence.lean` now supply the other direction. Start with the closure of the published bounded canonical orbit. Its compact dynamics admit an invariant probability, from which the proof constructs a forward recurrent point. The positive physical clock has a uniform lower speed bound; its inverse therefore defines a complete ordinary-time source trajectory and transports the forward return times. Continuity extends the validity identities and uniform state bound from the dense canonical orbit to the closure.

For each actual scalar drift coordinate, the published Fourier construction provides a bounded primitive on that closure. Before the clock change its derivative is the clock speed times the scalar

drift. Differentiation after the inverse clock change cancels that speed. Thus the ordinary-time integral is the difference of two bounded primitive values. Dividing by the full real horizon $H \to \infty$ proves a zero mean. No desired test mean is assumed. The uniform state bound also controls every nonnegative real moment at every positive horizon.

Combining this construction with the exclusion direction gives `exists_prime_source_trajectory_iff_not_solvable`: for distinct prime clocks, nonzero linear columns, and every fixed real $\mu \geq 1$, a complete valid forward-recurrent trajectory satisfying the growth and all scalar test conditions exists if and only if the integer system has no solution. The corrected compiler centerings equal the printed centerings under the nonzero-column hypothesis; that equality is proved for the literal scalar codes before combining the two directions.

This converse constructs an orbit-closure trajectory rather than the manuscript's literal torus graph Γ. It proves the source-level existence statement, but does not by itself identify all of that graph's weighted Fourier norms.

7.7 Extracting the trajectory from a compact-base measure

`OrdinaryTimeDetectorExtraction.lean` starts with an ergodic invariant probability for the actual compact base flow. It assumes full mass in the finite chart and zero integrals of every compact scalar drift, with unit gain. It then constructs a complete ordinary-time source trajectory, forward recurrent at zero, satisfying pointwise validity and all full real-horizon scalar mean conditions. The growth exponent is the actual even denominator-clearing exponent μ, and the bound holds at every horizon $H \geq 1$. The proof also checks $\mu \geq 1$.

The construction first restricts to the finite chart and selects a point that is both recurrent and generic for the needed observables. A positive clock change converts its compact orbit to an ordinary-time orbit. The clock-speed averages in both time directions ensure that this new orbit exists for all real times; no backward recurrence is assumed. A nonnegative validity-energy coordinate has zero integral, so continuity and invariance give its pointwise vanishing along the selected orbit. The other zero means survive the clock change by the corresponding ratio-of-integrals argument. An average estimate for the clock weight gives the Euclidean growth bound.

Thus the recurrence, validity, and ordinary-time averages are conclusions, not extra selection hypotheses. Combining this extraction with the source equivalence shows that a solvable arithmetic instance admits no such ergodic finite-chart probability. `OrdinaryTimeConditionalBase.lean` obtains precisely this base measure from an ergodic accumulator measure with positive finite-interior mass: restriction and the equivariant base projection preserve ergodicity, the factor has full finite-chart mass, and a nonzero drift gain cancels from the zero-mean equations.

`OrdinaryTimeAccumulatorGap.lean` combines the resulting solvable- instance obstruction with the published nonsolvable-instance finite graph. This proves the complete compact sign gap for the actual effective compiler, and its homogeneous wrapper is GAS exactly when the arithmetic instance is solvable. `OrdinaryTimeUniversalFamily.lean` supplies nonzero columns by the existing computable augmentation and gives fixed-pair and cofinal Σ_1^0-completeness on this proof route. The output code is definitionally the existing finite compiler; the selected measure is not an input to code generation.

This first construction retains the published effective gain $1/[8(T+1)(R+1)]$ and reward margin. The following modules separately implement Section 9's least-integer gain, chosen by $m^2 w_* \geq 4K_e^2$, and margin $w_*/4$. They prove the full torus-graph norm package and the two-sided invariant-average bounds for those literal constants. The two code families need not be equal; each has its own checked sign argument.

7.8 An actual weighted Fourier coordinate

`OrdinaryTimeWeightedFourier.lean` identifies the Haar Fourier coefficients of an absolutely summable character expansion by collecting all terms with the same frequency. Its weighted norm is bounded by the sum of the weighted expansion coefficients, even when frequencies collide. This avoids assuming that a formal expansion is already the Fourier series with distinct indices.

`OrdinaryTimeThetaGraph.lean` applies this result to the half-period theta coordinate T_1 of the literal torus graph. In fundamental clock coordinates, its n-th term has coefficient $4\lambda^{n^2}$, with $\lambda = 1/256$, and frequency length

$$|4n^2 - 1| + |4n - 2| \sum_{\ell} |B_{\ell j}|.$$

The function agrees with the shifted Jacobi value at every real lift of the torus phases, not just along one orbit. The corrected Gaussian bound proves summability of its actual weighted Haar coefficients whenever $\lambda\rho^4 < 1$ and $\rho \geq 1$. The module also computes the exact right-hand-side character norm $\rho^{4\sum_\ell |b_\ell|}$.

`OrdinaryTimeFourierInverse.lean` proves a reusable reciprocal estimate. If an absolutely summable character expansion h has weighted coefficient sum at most $\delta < 1$, expand $(1-h)^{-1} = \sum_{r\geq 0} h^r$. Each power is indexed by finite tuples of the original terms. Submultiplicativity of the weight bounds its sum by δ^r. Collecting equal frequencies only decreases the resulting upper bound, so the actual weighted Haar norm is at most $(1-\delta)^{-1}$. This proves a statement about the reciprocal function, not just a formal series with an assumed norm.

`OrdinaryTimePoissonGraph.lean` applies this argument to both signs of each Poisson reciprocal. The norm is at most 4 when the corresponding weighted geometric ratio is at most $3/4$. For the theta reciprocal, remove the leading gauge character and factor T_1 as 4 times a character times a centered theta function. Under

$$\frac{1}{256}\rho^{4+4\sum_\ell |B_{\ell j}|} \leq \frac{1}{8},$$

the nonconstant centered tail has weighted norm at most $2/7$. Its reciprocal therefore has norm at most $7/5$. Restoring the gauge character gives the literal coordinate $U = 1/T_1$ and the bound

$$\|U\|_\rho \leq \frac{7}{20}\rho^{1+2\sum_\ell |B_{\ell j}|}.$$

`OrdinaryTimeThetaReciprocal.lean` proves nonvanishing and the identity $T_1 U = 1$ at every phase. `OrdinaryTimeJacobiGraph.lean` also identifies all four Jacobi coordinates with explicit torus series and proves their actual weighted summability. The derivative, unary theta, and Eisenstein modules provide the remaining complex coordinates.

7.9 Assembling the graph and composing with polynomials

`OrdinaryTimeSourceGraph.lean` assembles these coordinates into the literal source-state type and its real coordinate encoding. It proves continuity of the resulting map Γ. The Fourier algebra modules work with actual Haar coefficients: products satisfy the weighted product bound, and real or imaginary parts do not increase the weighted norm. The latter fact follows from reflection of frequencies under complex conjugation. For example,

$$\widehat{\mathrm{Re}\, f}(a) = \frac{\widehat{f}(a) + \overline{\widehat{f}(-a)}}{2}.$$

Thus realification does not introduce an unnecessary factor of two.

`OrdinaryTimeSourceFourier.lean` combines the coordinate estimates. Every actual coordinate of Γ has summable weighted Fourier coefficients under the two printed matrix-norm conditions on ρ. Changing the arithmetic right-hand side b changes only its unit-modulus oscillator. In particular, the squared Euclidean norm of Γ is exactly independent of b, not just bounded uniformly in b.

The polynomial composition lemma has a useful general form. If each coordinate function has weighted norm at most $C \geq 1$, then

$$\|P \circ \Gamma\|_\rho \leq \left(\sum_e |P_e| \right) C^{\deg P}.$$

Expand P into monomials, apply the product bound to each monomial, and use $C \geq 1$ to replace its degree by $\deg P$. This proof includes constant and zero polynomials and does not require distinct frequencies in the coordinate expansions.

Separating the right-hand-side oscillator gives a natural constant C, uniform in b and P, such that

$$\|P \circ \Gamma_b\|_\rho \leq \left(\sum_e |P_e| \right) C^{\deg P} \rho^{4 \deg P \sum_\ell |b_\ell|}.$$

This statement is proved in `OrdinaryTimeSourceFourierBound.lean`. That first proof uses Archimedean existence. Three numerical modules now give an explicit choice:

- `OrdinaryTimeThetaBudget.lean`: scalar theta estimates;
- `OrdinaryTimeSourceBudget.lean`: all graph coordinates;
- `OrdinaryTimeSourcePolynomialBudget.lean`: polynomial substitution.

Their common coordinate bound is

$$C = 262144^4 + 1048576 + 385 + 32 + 1.$$

This deliberately coarse bound is independent of the arithmetic input. The scalar estimates bound each Jacobi theta coordinate by 262144, its normalized derivative by 1048576, and the Eisenstein coordinate by 385. Polynomial powers, reciprocal coordinates, and the real coordinate encoding then give the displayed C. These estimates retain the signed shift in the theta exponent, including its negative values.

Fourier reconstruction bounds the absolute value of each coordinate by its weighted coefficient norm. Applying this at $\rho = 1$ gives the explicit squared Euclidean bound

$$\|\Gamma_b(x)\|_2^2 \leq NC^2,$$

where N is the source-state dimension. The normalized positive controller equilibria satisfy the required coordinate bounds because their two squared coordinates sum to one. Thus neither this size bound nor the polynomial Fourier bound relies on a non-effective choice of constant.

7.10 An explicit primitive bound

Let the torus have r coordinates with distinct prime clocks p_i and put $s = 2^r - 1$. At a radius $\rho = 1 + 1/m$, $m \geq 1$, a usable natural small-divisor multiplier is

$$B_{\mathrm{rad}} = \left(1 + \sum_i p_i \right)^s s!(2m)^s.$$

The algebraic small-divisor estimate costs a polynomial in the integer frequency. The elementary binomial estimate

$$(1+n)^s \le s!(2m)^s(1+1/m)^n$$

absorbs that cost into the exponential Fourier weight. This includes $r=0$, $s=0$, and $n=0$.

For a real polynomial family (P_j) of degrees at most D, define

$$K_0 = 1 + B_{\mathrm{rad}} \left(\sum_j \sum_e |(P_j)_e| \right) C^D.$$

The literal Fourier primitive H is continuous and real-valued, and

$$\|H(x)\|_2^2 \le \left(K_0 \rho^{4D \sum_\ell |b_\ell|} \right)^2.$$

The estimate first bounds the sum of the coordinate absolute values, then uses $\sum_j |H_j|^2 \le (\sum_j |H_j|)^2$. No extra dimension factor is concealed in K_0.

Along the physical signed-prime-root flow, its derivative is exactly $P_j \circ \Gamma_b - \widehat{P_j \circ \Gamma_b}(0)$. The normalization of the torus frequency by $1/(2\pi)$ cancels the 2π in differentiating a character. Conjugate symmetry of the actual Haar coefficients proves that the primitive is real. The constant coefficient is retained until its arithmetic vanishing is established; it is not an implicit premise of the primitive theorem. These results are checked in `OrdinaryTimeRadicalFourierBudget.lean`, `OrdinaryTimeRadicalPrimitive.lean`, and `OrdinaryTimeSourcePrimitive.lean`.

7.11 The rational radius and least gain denominator

The radius search does not compare arbitrary real numbers. For $\rho = 1 + 1/m$, its two tests are equivalent to the integer inequalities

$$(m+1)^{K_B} \le 32 m^{K_B}, \qquad 2(m+1)^{K_A} \le 3m^{K_A}, \qquad m > 0.$$

The rational powers tend to one as $m \to \infty$, so a successful test exists. The module `OrdinaryTimeRadiusChoice.lean` proves that searching for the least successful integer is a total computable operation. It also proves that the resulting radius satisfies the matrix-norm inequalities used above.

For the actual decoded detector drift, the modules `OrdinaryTimeDetectorResources.lean` and `OrdinaryTimeDetectorGain.lean` define the semantic degree D_h, the positive rational K_0, and $K_e = K_0 \rho^{4D_h \sum_\ell |b_\ell|}$. The drift family, its degree, and K_0 are unchanged when only the arithmetic right-hand side changes. Set $M_0 = NC^2$ and $w_* = 1/(1+M_0)$. The gain module implements the least positive integer k such that $4K_e^2 \le k^2 w_*$, using integer comparisons on a rational code. It proves both leastness and computability, and obtains

$$\kappa = 1/k > 0, \qquad \kappa^2 K_e^2 \le w_*/4, \qquad \varepsilon = w_*/4 > 0.$$

On the source and scaled-primitive graph, write $z = \kappa H(x)$. The central coordinate satisfies $w^2 \ge w_*$, while $|z|^2 \le w_*/4$. Consequently the literal stereographic reward obeys

$$\frac{1-|z|^2}{1+|z|^2} + 2w^2 - 1 - \varepsilon \ge 2w_* - 2|z|^2 - \varepsilon \ge \frac{5}{4} w_*.$$

This is a pointwise bound on the specified graph. Its dynamical use requires the separate graph-invariance and arithmetic zero-mode results. The corresponding input algorithm is checked in

`OrdinaryTimeDetectorGainCode.lean`. With the matrices and prime tuple fixed, it computes the budget code primitive-recursively from the integer right-hand side, then applies the computable least-denominator search. Equal decoded rational budgets give the same least denominator, so the proof does not require the intermediate rational codes to be in reduced form.

7.12 The graph ODE and its arithmetic zero mode

The explicit graph must solve the source ODE, not just satisfy its Fourier estimates. The source-flow modules identify every coordinate of $\Gamma(\varphi + \omega t)$ with the established canonical trajectory at the same initial phase. This includes the gauge, both reciprocal signs, and the right-hand-side oscillator. The identity $T_1 U = 1$ proves the required nonvanishing chart condition. For the positive prime controllers, their angular velocity is exactly $\sqrt{p_i}$; its coordinate in fundamental turns is $\sqrt{p_i}/(2\pi)$. Thus the graph equation uses the actual polynomial vector field and physical time.

To identify the constant coefficient, assume the arithmetic instance is unsolvable. The previously checked arithmetic Fourier expansion gives a bounded primitive for each actual drift coordinate on the zero-phase orbit, so its full forward time average is zero. Independently, the new actual-Haar primitive has derivative $f - \widehat{f}(0)$ and is bounded. Its derivative also has zero forward mean. Subtracting these two identities forces $\operatorname{Re} \widehat{f}(0) = 0$; reality of f gives the imaginary part as zero. This proves vanishing of the actual Haar coefficient, not merely of a coefficient in a formal expansion. Consequently the primitive equation is uncentered at every initial phase.

The graph-flow endpoint is in `OrdinaryTimeDetectorGraphFlow.lean`. The coefficient calculation is in `OrdinaryTimeDetectorZeroMode.lean`. Neither assumes the trajectory-selector tests or a previously selected invariant measure.

7.13 The compact graph and its invariant probability

Write H for the uncentered primitive just constructed and $z = \kappa H$. The compact graph is the image of the torus under

$$x \longmapsto \left(\frac{(\Gamma(x), 1)}{\sqrt{1 + |\Gamma(x)|^2}}, \frac{(2z(x), 1 - |z(x)|^2)}{1 + |z(x)|^2} \right).$$

The first factor is positive central compactification; the second is inverse stereographic projection. The image is nonempty and compact, because the torus is nonempty and compact and both maps are continuous.

Compactness alone does not make this image invariant. The proof follows an actual source orbit $a(s) = \Gamma(x + \omega s)$ and uses the reciprocal central clock below. Here $2K$ is the clearing exponent; in the detector specialization, K is `sourceDegreeBoundI`, not the primitive budget K_e.

$$t(s) = \int_0^s (1 + |a(u)|^2)^K \, du.$$

Its derivative is at least one, so this clock is an increasing diffeomorphism of the whole real line. In compact time, both a and z acquire the same factor $(1 + |a|^2)^{-K}$. Differentiating the central and stereographic chart maps therefore gives the literal compiled polynomial vector field. Uniqueness identifies this curve with the canonical compact flow. Thus that flow preserves the graph image.

This reasoning does not assume that Γ is injective, or that H is a function of Γ. Both are functions of the original torus parameter. Nor does it assert equivariance in unchanged time: the reciprocal clock is part of the proof.

Restrict the compact flow to this invariant image and apply Krylov–Bogolyubov. Pushing the resulting invariant probability into the ambient compact source gives the graph full mass. Since the literal reward is at least $5w_*/4$ at every graph point, its integral is at least $5w_*/4 > 0$. In particular, the maximal invariant average β is positive for an unsolvable arithmetic instance.

The proof is divided into four reusable steps:

- `Dynamics/CompactInvariantGraph.lean`: the measure theorem;
- `OrdinaryTimeCompactGraphTrajectory.lean`: chart differentiation and flow uniqueness;
- `OrdinaryTimePositiveGraph.lean`: reciprocal-clock invariance;
- `OrdinaryTimeDetectorInvariantGraph.lean`: the actual detector.

The last step uses the raw least-denominator gain code and its proved decoding identity.

7.14 The negative branch uses the same field

The module `OrdinaryTimeDetectorGap.lean` proves the negative branch: $\beta \leq -\varepsilon$ for a solvable arithmetic instance with the stated nonzero-column condition. The argument works for every nonzero rational gain and every rational margin. A hypothetical ergodic invariant probability with positive finite-chart mass gives an ergodic conditional base probability. If all its drift means vanished, ordinary-time extraction would imply arithmetic unsolvability. Thus some conditional drift mean is nonzero. The compact-accumulator obstruction then forces an ergodic maximizing measure outside the finite chart, onto the central boundary or accumulator infinity. There the literal reward is at most $-\varepsilon$.

Specializing to the same raw gain code and the margin $w_*/4$ proves the negative branch for the field used in the positive graph argument. No identification with the earlier alternative gain is needed. Both branches concern the actual compiled flow and reward, rather than an abstract sign-gap interface supplied as a hypothesis.

7.15 The literal gain feeds a computable homogeneous compiler

The sign theorem and the code generator must describe the same field. The module `OrdinaryTimeDetectorSign.lean` packages the two branches into a semantic certificate and proves that its source field and reward are literally the existing sparse-polynomial compiler applied to the new raw gain and margin codes. The faithful homogeneous wrapper is then GAS exactly when the arithmetic instance is solvable.

With the matrices and prime tuple fixed, replacing the integer right-hand side is primitive recursive. Computing the least-admissible gain denominator is total computable. Composing these operations with the polynomial compiler and homogeneous wrapper gives a computable field-code map. Its definition takes no proof of solvability, graph invariance, or the sign theorem. The module `OrdinaryTimeDetectorCompiler.lean` proves both its computability and its literal equality to the field carried by the semantic certificate.

Finally, `OrdinaryTimeUniversalDetector.lean` applies this map to the fixed augmented universal DQL template. Only the right-hand side varies; the augmentation supplies the required nonzero linear columns. All field dimensions and degree bounds are therefore fixed. This gives an exact code-level equivalence between universal halting and HGAS for the printed-gain family. The preserved RE upper bound and cubicization and padding theorems give fixed-pair and cofinal Σ_1^0-completeness. This composition does not identify the printed gain with the earlier alternative gain; it verifies a separate effective family using the same established polynomial compiler and wrapper.

8 A fixed computational source and register instructions

The module `Computability/UniversalMachineSource.lean` starts with Mathlib's partial-recursive universal evaluator, not an assumed universal register machine. A pair consisting of a program code and its input has a primitive-recursive natural-number encoding. The list-code completeness theorem supplies one fixed program computing that evaluator on singleton lists. Mathlib's verified translation then gives a fixed stack-machine interpreter. Its halting predicate agrees with `HGAS.UniversalHalts` under the input encoding, so its halting language is Σ_1^0-complete.

The compiled interpreter has one finite control support independent of the input. This support is derived from the compiler and contains every reachable active label. The interpreter code is chosen from a proved existence theorem; it is not an evaluated numeric program listing.

Separately, `Computability/RegisterMachine.lean` defines finite increment/decrement programs with natural-valued registers, finitely many active labels, and an absorbing halting label. Its total finite-step run halts exactly when Mathlib's partial transition evaluator is defined. The clear-register loop takes $n + 1$ steps when the register initially holds n: n decrements and a final zero test. The transfer loop takes $2n + 1$ steps: each decrement is followed by a target increment, then a final zero test. The formal equations preserve all other registers.

The module `RegisterMachineArithmetic.lean` implements the arithmetic needed for numeral stacks. Symbols use the nonzero digits $1, \ldots, b - 1$, with the top at the least significant digit; the empty stack has code zero. Division by b removes the top, remainder reads its digit, and the encoding is injective. Four symbols can therefore use base five.

To push digit d, decrement the value register once and increment a cleared scratch register b times, repeating until the value is zero. Transfer the scratch back, then increment d times. This produces $bN + d$ and clears the scratch. To pop, cycle through b decrement/zero-test labels, incrementing the quotient scratch after each complete cycle. The test at which zero is found records the remainder in finite control. Its continuation transfers the quotient back and clears the scratch. The correctness theorems derive these runs from literal instruction equations and preserve every other register. They include the final zero tests in their exact step counts; they do not assume the desired macro behavior as part of an interface.

The modules `RegisterMachineStackBlocks.lean` and `RegisterMachineDispatch.lean` now allocate finite push/pop blocks and finite store-dispatch tables. Mathlib's interpreter uses four stacks, four symbols, and a five-valued optional-symbol store. Four stack registers and one cleared scratch register suffice for these operations. The store selects an initial control label in a fixed program; it does not select a different program at runtime. A push block has 12 internal labels and a pop block 16; allocating all five store branches gives 60 and 80 labels, respectively. The proofs match the literal Mathlib instruction semantics, including store updates and continuation labels.

`RegisterMachinePeek.lean` implements peek by pop followed by restoration, retaining both the old store and popped symbol in finite control. Its proof restores every stack and clears the scratch register, including the empty-stack case. `RegisterMachineEmbedding.lean` then allows a block's exit labels to refer to any ambient program labels, including backward cyclic jumps. The proof reindexes an already verified block whose continuation is the ambient program itself. Only literal local instruction equations are required; a simulation property is not assumed.

`RegisterMachineStatement.lean` recursively compiles all seven constructors of Mathlib's actual stack-statement syntax. Both arms of a branch receive fixed finite label ranges. A structural induction proves that each complete statement runs for a positive finite number of increment/decrement steps and reaches exactly the result of `TM2.stepAux`, with its finite store encoded in the exit label and the scratch register cleared. The external destination instruction has not yet executed. This is an actual compiler and semantic theorem, not an assumed interpreter interface.

`RegisterMachineFinite.lean` assembles these statements into one finite cyclic program. The block width is computed from the maximum statement size, so neither a size bound nor a simulation relation is assumed. Its halting theorem compares the actual register run with Mathlib's partial stack-machine evaluator in both directions. `RegisterMachineSupport.lean` restricts the interpreter's control labels to its proved finite support. Its arbitrary fallback outside that support only makes the encoding total; reachable configurations never use it.

Finally, `RegisterMachineInput.lean` encodes the actual binary input stack. The primitive-recursive numeral map satisfies

$$F(0) = 2, \qquad F(n) = 5F(\lfloor n/2 \rfloor) + 3 + (n \bmod 2) \quad (n > 0).$$

All other stack registers and the scratch register initially vanish. `RegisterMachineUniversal.lean` therefore obtains one fixed finite five-register INC/DEC program, with the input in register zero, and an explicit primitive-recursive reduction from `HGAS.UniversalHalts` to its literal halting predicate. Universality is now a conclusion of the assembled compiler.

The membership argument applies to every raw input numeral, not only encoded source inputs. `RegisterMachineComputability.lean` proves that a finite program's packed step and finite run are primitive recursive; searching for a halting label semidecides its halting language. Together with the reduction, this proves Σ_1^0-completeness of the one fixed register program's input language. The later monitor compiler and `UniversalStabilityReduction.lean` connect this language to HLS.

9 Restart budgets

The following finite inequality is the counting argument in [2, Proposition 4.5, Step 3]. It is the first completed component of the new proof-based formalization of that reduction.

Lemma 9.1. *Let h_j be real budgets, let $e_j \geq 0$ be errors, and distinguish some turns as productive. Suppose, for $j < n$,*

$$h_{j+1} \geq h_j - e_j, \qquad h_{j+1} \geq \max(h_j + 1, 1) - e_j \quad \textit{when turn } j \textit{ is productive.}$$

If $P_n > 0$ is the number of productive turns before n, then

$$h_n \geq P_n - \sum_{j<n} e_j.$$

In particular, if $P_n \geq T \in \mathbb{N}$ and $\sum_{j<n} e_j < 1$, then $\max(h_n, 0) + 1 > T$.

Proof. At the first productive turn the reset to at least one removes all dependence on the initial budget, which may be negative. Each later productive turn adds at least one before its error; every other turn loses at most its error. Induction gives the displayed bound. If $P_n > 0$, combine it with $P_n \geq T$ and the total-error bound. If $P_n = 0$, then $T = 0$, and $\max(h_n, 0) + 1 \geq 1 > 0$. □

The exact finite-prefix statement is `Stability.Simulation.productiveCount_sub_error_le_budget`, in `Stability/Simulation/RestartBudget.lean`. It does not assume a simulation-correctness package. The clocked component estimates below now supply the corresponding one-sided ODE inequalities; the full monitor still needs their connection to its output events and discrete halting runs.

9.1 Literal discrete restart runs

`RegisterRestart.lean` defines the restart machine itself, with a natural budget, integer countdown, natural registers, and a restart, active, or halt label. A restart increases the budget by one, reloads the input, and begins a clean attempt. Its budget after any finite run equals its initial budget plus the actual number of resets.

If the input machine halts within T steps, an attempt funded with at least T units reaches halt before it can restart. Therefore every finite run, from every state in this discrete domain, has at most T reset labels: by the Tth reset the newly granted budget is sufficient, and no later reset is possible. This also covers arbitrary initial registers and negative integer countdowns.

Conversely, if the input never halts, each clean funded attempt exhausts its countdown and returns to restart. Starting at the designated zero budget restart state gives arbitrarily many resets. `RegisterRestartNonhalting.lean` proves this construction and the equivalence between input halting and a uniform finite reset-count bound. It does not claim unbounded counts from every initial state: a state already at halt is an immediate counterexample to such a claim.

`RegisterRestartEmbedding.lean` proves that the explicit real transition map intertwines with every discrete step and hence every finite iterate. The reset-count equivalence therefore holds on the embedded discrete domain. Extending the required tracking estimates to arbitrary real states of the continuous monitor is a further task.

10 A finite-interval cubic relaxation estimate

The first analytic component of the simulation proof is a scalar relaxation estimate [2, Lemma 4.1]. The completed module `Stability/Simulation/CubicRelaxation.lean` proves its strictly positive-excess branch as a differential inequality, not as an assumed simulation property.

Lemma 10.1. *Let $a \le b$. Suppose w and γ are continuous on $[a,b]$, $w > 0$ there, and w is differentiable in (a,b) with*

$$w'(t) \le -\gamma(t) w(t)^3.$$

Then

$$\frac{1}{w(a)^2} + 2\int_a^b \gamma(t)\,dt \le \frac{1}{w(b)^2}.$$

If $\int_a^b \gamma > 0$, it follows that

$$w(b) < \left(2\int_a^b \gamma(t)\,dt\right)^{-1/2}.$$

Proof. Since $w > 0$, differentiation gives $(w^{-2})' = -2w'/w^3 \ge 2\gamma$ in the interior. The integral comparison theorem bounds the integral of the continuous lower derivative bound by the endpoint difference of w^{-2}. This proves the first inequality without requiring continuity of w'. The initial reciprocal is strictly positive. Dropping it therefore gives a strict inequality, which yields the second assertion after multiplying by $w(b)^2$ and taking square roots. □

The reciprocal inequality does not require $\gamma \ge 0$; only the final inverse-square-root estimate needs positive integrated gain. The important restriction is $w > 0$ on the whole interval. Handling zeros, moving targets, additive forcing, and continuation requires further arguments. These theorems are not labeled as a formalization of the entire relaxation lemma.

10.1 A constant target, including sign changes and zeros

The companion module `Stability/Simulation/ConstantTargetCubicRelaxation.lean` handles an actual unforced solution segment

$$y' = \gamma(t)(A - y)^3, \qquad \gamma \geq 0,$$

with constant target A, without any restriction on the initial value. The squared error satisfies

$$\frac{d}{dt}(y - A)^2 = -2\gamma(t)(y - A)^4 \leq 0.$$

Consequently, reaching the target at any time forces the solution to stay there for the rest of the interval. If $y(b) \neq A$, the error cannot have vanished earlier. Its absolute value is therefore differentiable throughout the interior and satisfies

$$\frac{d}{dt}|y - A| = -\gamma(t)|y - A|^3.$$

The positive-excess lemma applies. If $y(b) = A$, the same final bound is immediate. Thus, for positive integrated gain,

$$|y(b) - A| < \left(2\int_a^b \gamma(t)\,dt\right)^{-1/2}.$$

This proof uses no separate uniqueness theorem. It assumes a solution segment is already defined. The next two modules extend the estimate to moving targets and forcing; existence remains a separate obligation.

10.2 Moving targets and additive forcing

Suppose the supplied scalar solution satisfies

$$y' = \gamma(t)(A(t) - y)^3 + \varphi(t), \qquad \gamma(t) \geq 0, \qquad |A(t) - a_*| \leq r,$$

on a finite interval $[a, b]$, with continuous rate and forcing and with positive integrated gain. Define the cumulative forcing mass

$$\Phi(t) = \int_a^t |\varphi(s)|\,ds.$$

For the upper excess $w(t) = y(t) - (a_* + r) - \Phi(t)$, differentiation and $\varphi - |\varphi| \leq 0$ give $w' \leq -\gamma w^3$ whenever $w > 0$. The squared positive part $(\max(w, 0))^2$ is differentiable even at $w = 0$, and its derivative is nonpositive. Thus an excess cannot cross from nonpositive to positive. If its final value is positive, it was positive throughout and the reciprocal-square estimate applies. If its final value is nonpositive, the desired upper bound is immediate. Applying the same argument to $(a_* - r) - y - \Phi$ yields

$$|y(b) - a_*| < r + \left(2\int_a^b \gamma(t)\,dt\right)^{-1/2} + \int_a^b |\varphi(t)|\,dt.$$

There is only one forcing-mass term, not two.

The same no-crossing argument, without a positive-gain hypothesis, proves

$$|y(t)| \leq \max\{|y(a)|, R\} + \int_a^b |\varphi(s)|\,ds \quad (a \leq t \leq b)$$

whenever $|A(t)| \leq R$. One-sided target bounds give the corresponding one-sided relaxation estimates. These results are in `MovingTargetCubicRelaxation.lean` and `ForcedCubicRelaxation.lean` under `Stability/Simulation/`. They formalize the finite-segment estimates in [2, Lemma 4.1], but do not themselves construct a solution or prove an ODE continuation theorem.

11 Clock gain and a uniform summable error budget

The module `Simulation/ClockGain.lean` starts with the actual clock equations on a supplied finite interval:

$$\theta' = 1, \qquad u' = -v, \qquad v' = u.$$

The first equation gives $\theta(t) = \theta(a) + t - a$. Differentiating the two rotating coordinates gives

$$u(t) = u(a)\cos(t-a) - v(a)\sin(t-a),$$
$$v(t) = u(a)\sin(t-a) + v(a)\cos(t-a).$$

In particular, $u^2 + v^2$ is constant. These are identities for supplied solution segments, not assertions of global existence.

On a centered active arc, put

$$\gamma(t) = K(1 + (c+t)^2)^2 \max(r\cos t, 0), \qquad |t| \le \pi/3,$$

where $K > 0$, $r \ge 0$, and $r^2 > 1/2$. Set $d = \max(|c| - \pi/3, 0)$. Then $|c + t| \ge d$, while $r > 2/3$ and $\cos t \ge 1/2$ give $\max(r\cos t, 0) \ge 1/3$. Integrating this pointwise estimate, using $4\pi > 9$, yields

$$K(1+d^2)^2 \le 2\int_{-\pi/3}^{\pi/3} \gamma(t)\,dt.$$

Thus the relaxation error from the preceding section is at most

$$K^{-1/2}e(c), \qquad e(c) = \frac{1}{1 + \max(|c| - \pi/3, 0)^2}.$$

The module `Simulation/ClockErrorSummability.lean` proves the uniform bound

$$\sum_{j\in\mathbb{Z}} e(c + j\pi) \le \frac{29}{10} < 3 \qquad (c \in \mathbb{R}).$$

To see the estimate, shift c by an integer multiple of π into $[0, \pi]$. Split the integer lattice into $j = n$ and $j = -(n+1)$. On either half, the distance of the nth point from zero is at least $n\pi$, so its error is at most $a_n = (1 + 4n^2)^{-1}$. Here $a_0 = 1$, $a_1 = 1/5$, and for $n \ge 2$,

$$a_n \le \frac{1}{4n(n-1)} = \frac{1}{4(n-1)} - \frac{1}{4n}.$$

The tail telescopes to at most $1/4$, giving $\sum_{n\ge 0} a_n \le 29/20$ on each half. Summability is established before manipulating infinite sums. The forward lattice and every finite subset inherit the strict bound 3, and multiplying by $K^{-1/2}$ gives the error budget in [2, Lemma 4.2]. Identifying these centers with every turn of the actual monitor, including initial partial turns, requires further bookkeeping.

The module `AlternatingClock.lean` supplies this connection for an explicitly aligned phase $u(t_0) = 0$, $v(t_0) = -r$. Its half-turn boundaries are $\tau_n = t_0 + n\pi$, and their centers are $c_n = \tau_n + \pi/2$. The oscillator equation gives

$$(-1)^n u(c_n + s) = r\cos s, \qquad \theta(c_n) = \theta(t_0) + \pi/2 + n\pi.$$

The opposite buffer's gate vanishes throughout the corresponding half-turn. The actual gain in centered coordinates is therefore the gain function estimated above. Half-open half-turns are pairwise disjoint.

The module `HalfTurnForcingMass.lean` proves that the absolute forcing integrals over the first M half-turns sum exactly to the integral over $[t_0, \tau_M]$. Combining this partition with the clock series gives, in `HalfTurnErrorBudget.lean`,

$$\sum_{n<M} \left(2 \int_{\tau_n}^{\tau_{n+1}} |\varphi(t)|\, dt + K^{-1/2} e(\theta(c_n)) \right) < 2 \int_{t_0}^{\tau_M} |\varphi(t)|\, dt + 3K^{-1/2}.$$

For $K = 2^{20}$ and total forcing mass at most $1/256$, this is less than $11/1024$.

11.1 The actual compute/copy update

Three modules connect the allowances to the actual two-buffer dynamics:

`DoubleBufferUpdate.lean`
`HalfTurnGain.lean`
`ClockedDoubleBufferUpdate.lean`.

During a compute half-turn, the copy gate vanishes, so the held buffer a moves only by its forcing. Write E_c for the integral of the larger of the two forcing norms during this half-turn. Then $\|a(t) - a(t_0)\|_\infty \le E_c$.

Suppose the real transition map agrees along this compute path with a map F of Lipschitz constant at most one. Its moving target stays within E_c of $F(a(t_0))$. The forced cubic-relaxation estimate gives

$$\|b(t_1) - F(a(t_0))\|_\infty \le 2E_c + \rho_c, \qquad \rho_c = \left(2 \int_{t_0}^{t_1} \gamma_c \right)^{-1/2}.$$

During the following copy half-turn, b is held and moves by at most its forcing budget E_p. Applying the same estimate to a gives

$$\|a(t_2) - F(a(t_0))\|_\infty \le 2(E_c + E_p) + \rho_c + \rho_p.$$

There is no hypothesis on the initial distance of b from its target.

The whole-half-turn gain is at least the central-arc gain, by positivity and interval inclusion. The clock identities therefore bound ρ_c and ρ_p by the corresponding lattice errors. The final theorem starts with the oscillator-gated component ODEs and concludes that one update error is at most the sum of its two half-turn allowances. It does not assume vanishing gates, positive integrated gain, or the update estimate.

11.2 The real transition map, budgets, and clean resets

The real transition map is now explicit in `RegisterTransition.lean`. It uses a real budget, countdown, label, and register coordinates. Integer active labels are encoded by $1, \dots, s$, restart by zero, and halt by $s + 1$. Piecewise-linear gates select the instruction branch and distinguish zero from positive registers. On the stated regular neighborhoods of radius $1/8$, these gates are constant. Freezing them gives an affine branch of Lipschitz constant at most one in the sup norm. The reset branch has the same bound because $h \mapsto \max(h, 0) + 1$ is nonexpansive; the halt branch fixes honest halted states. These facts are proved in `RegisterLocalBranches.lean`.

`RegisterTransitionExactness.lean` checks the discrete meaning: with natural registers and countdown at least one, an active branch executes the literal increment/decrement instruction. With countdown at most zero it preserves the registers and selects restart. Restart sets both budget and

countdown to $\max(h, 0) + 1$, restores the input registers, and selects the entry label. No claim about every fractional countdown is hidden in these exactness statements.

`RegisterTransitionFormula.lean` expresses each coordinate of the same map by an explicit absolute-value formula and proves equality of their evaluations. Finite sums retain every branch, including zero terms. Changing the input natural numbers therefore changes only rational constant values, not the erased formula tree, radical count, or majorant degree bound. This is the syntactic resource theorem used later to prove primitive recursiveness of the raw-code compiler.

The modules `DoubleBufferBudget.lean` and `ClockedBudget.lean` derive one-sided budget bounds directly from the component ODEs. The ordinary bound uses the lower target $f(h) = h$; the reset bound uses $f(h) = \max(h + 1, 1)$. Both functions have Lipschitz constant one, so the same forcing and gain allowances apply.

Finally, the held buffer moves by at most one forcing mass between any two times in a compute interval. If that mass is at most $1/16$ and the label lies in $(-1/16, 1/16)$ once, it stays in $(-1/8, 1/8)$ throughout the interval. The real map is then the reset map along the entire path, and the compute/copy estimate proves a clean reset. `HeldBufferMotion.lean` and `DoubleBufferReset.lean` contain this argument.

11.3 The constructed monitor and finite-phase bounds

`ClockedMonitor.lean` assembles the actual field, with $K = 2^{20}$, and output

$$o = \max\{1 - 2|u^2 + v^2 - 1|, 0\} \max\{u, 0\} \lambda(4a_{\text{label}}),$$

where λ is the compactly supported label gate. Its positivity implies $1/2 < u^2 + v^2 < 3/2$, $u > 0$, and $|a_{\text{label}}| < 1/16$. Thus the amplitude and reset-label restrictions used above follow from an actual output event. The module `ClockedMonitorFormula.lean` represents every field coordinate and the output in the common absolute-value syntax, with input-independent erased formula trees. `ClockedMonitorDynamics.lean` applies the update and clean-reset estimates to the actual vector ODE with additive buffer forcing.

`FundedRealRun.lean` handles real, not necessarily integer, countdowns during a sufficiently funded clean attempt. If $B \geq T$, the first T iterates follow the actual register computation, with unused countdown at least $B - T$. If the computation halts, the reached real state is fixed. No claim is made that an exhausted fractional countdown continues to represent an honest machine state.

`FundedMonitorTracking.lean` uses the finite compute/copy bounds to track such a funded real attempt. The accumulated gain allowances and actual forcing mass keep the sampled error below the regular-branch radius. Thus each local branch used in the induction is justified by the preceding error bound, rather than supplied as an extra hypothesis. `HaltingMonitorTracking.lean` continues the comparison after the source halts: the reference state is then fixed, so no further countdown funding is needed. The held-buffer estimate also excludes positive output throughout each post-halt turn, not only at sample times.

This yields an all-initial-buffer event count. Call a complete turn an event if the actual output is positive somewhere during it. Each event forces a clean reset. The ordinary budget recurrence prevents substantial loss between events, while the productive recurrence increases the budget at an event. If the source halts after T instructions and the entire segment's forcing mass is at most $1/256$, a $(T + 1)$st event supplies a sufficiently funded reset. Tracking from that reset excludes any later event. Hence at most $T + 1$ turns are events. No initial buffer is assumed to encode a machine state. The phase is aligned, but the required amplitude restriction follows from any alleged event itself. `MonitorEvents.lean` and `MonitorEventBound.lean` prove this statement for the actual forced ODE and actual output.

The output is nonnegative and at most 2. Integrating over complete turns therefore costs at most 4π per event. Aligning an arbitrary phase discards at most two partial turns, giving the bound $4\pi(T+3)$ whenever total forcing mass is at most $1/256$. For arbitrary forcing, cut the interval whenever its accumulated mass reaches $1/256$. Continuity of this actual cumulative integral supplies the cuts by the intermediate value theorem. Adjacent-interval additivity then gives

$$\int o\,dt \le 4\pi(T+3)(256M+2),$$

where M is the actual forcing mass. `MonitorOutputIntegral.lean` and `MonitorForcingIntegral.lean` prove these bounds without any assumption on initial buffers, phase, amplitude, or segment length.

For nonhalting, start the held buffer at the designated integer restart state and use unit oscillator amplitude. The other buffer may be arbitrary. `IntegerMonitorTracking.lean` proves the real map's local branches at every embedded integer restart state and tracks every finite prefix under the small total forcing budget. On a reference reset turn, the held label stays in the region where the output gate equals one. The actual output is then the oscillator sine throughout the compute half-turn, whose integral is 2. Thus the integral is at least twice the exact reference reset count. That count is unbounded for nonhalting. The genuine global smoothed solution has forcing mass less than $\epsilon\pi < 1/256$ when $0 < \epsilon \le 1/1024$; hence this lower bound applies to it. `NonhaltingMonitorOutput.lean` constructs the resulting global witness rather than assuming its existence.

The unsmoothed field is locally Lipschitz, although absolute values need not be differentiable. `FormulaLocalExistence.lean` proves this regularity structurally and applies local ODE existence through every state. `OscillatorPhase.lean` finds an aligned phase on any supplied full period, including the zero-amplitude case. For shorter segments, `MonitorPhaseBounds.lean` needs only a constant sign of u. On a compute segment, a moves by at most the forcing mass. The actual continuous transition map H is bounded on the resulting compact ball, and cubic relaxation bounds b by the larger of its initial norm and that target bound, plus forcing mass. On a copy segment, the roles are reversed. Hence a uniform bound for either kind of segment can be chosen from the initial buffer bound and forcing-mass cap before seeing the trajectory. `OscillatorPartition.lean` chooses a sine origin in the period preceding the initial time and clips its half-turn grid to the supplied interval. Every resulting segment has constant sign, even for zero amplitude or a degenerate clipped segment. The number of pieces can be fixed from a time-horizon cap. Iterating the uniform phase bound gives, in `MonitorFiniteBounds.lean`, a full-state bound chosen from the time horizon, initial norm, and forcing-mass cap before the trajectory. No future solution is assumed to select this partition.

`BoundedContinuation.lean` turns these a priori estimates into existence. For a prescribed horizon, multiply a locally Lipschitz field by a Lipschitz norm cutoff which is one on a ball larger than the proved bound and zero outside a slightly larger ball. The cutoff field is bounded and globally Lipschitz, so it has a global trajectory. If that trajectory first reached the inner boundary, its preceding segment would solve the original equation and violate the a priori bound. It therefore solves the original equation throughout the horizon. Local uniqueness makes the trajectories constructed for different horizons agree, allowing them to be joined into a global forward trajectory. The proof does not require the original field to be differentiable.

`MonitorCompleteness.lean` applies this theorem to the actual unsmoothed monitor, for every real initial state. Thus forward existence is now proved, not inferred from a priori estimates alone. `SmoothedMonitorCompleteness.lean` then proves forward completeness for the actual shared-precision smoothing. Smoothing leaves the clock and oscillator formulas unchanged. Its remaining residual is a continuous buffer forcing whose joint mass is less than $\varepsilon\pi$ on every finite solution segment. The

same a priori bound and continuation theorem therefore apply. Complete robust tracking remains separate.

12 Smoothing formulas with polynomial error bounds

An absolute-value formula is built from rational constants and coordinates using addition, multiplication, and absolute value. For $L \geq 1$, replace each absolute-value node from the inside out by $t \mapsto \sqrt{t^2 + L^{-2}}$. Denote the original and smoothed values of a formula E by $E(x)$ and $\widehat{E}_L(x)$.

Theorem 12.1. *The explicit structural recursion below constructs rational polynomials U_E, V_E, independent of L, such that pointwise*

$$U_E, V_E \geq 0, \qquad |E|, |\widehat{E}_L| \leq U_E, \qquad |\widehat{E}_L - E| \leq V_E/L.$$

Proof. A coordinate receives $(U, V) = (1 + x_i^2, 0)$; a rational constant c receives $(1 + |c|, 0)$. For a sum, add both bounds. For a product, use

$$U = U_1 U_2, \qquad V = V_1 U_2 + U_1 V_2,$$

and the identity

$$\widehat{E}_1 \widehat{E}_2 - E_1 E_2 = (\widehat{E}_1 - E_1)\widehat{E}_2 + E_1(\widehat{E}_2 - E_2).$$

For an absolute-value node, use $(U, V) = (U_1 + 1, V_1 + 1)$. Indeed,

$$|t| \leq \sqrt{t^2 + L^{-2}} \leq |t| + L^{-1}, \qquad L^{-1} \leq 1,$$

and $||s| - |t|| \leq |s - t|$. These inequalities establish each inductive step, including nonnegativity of the two majorants. □

The theorem is `AbsFormula.majorants_correct` in `Stability/Simulation/FormulaSmoothing.lean`; it follows [2, Lemma 5.1]. Since L is universally quantified at each point, the same statement applies when the precision depends on the state.

The module `FormulaResources.lean` bounds the degrees of both majorants by a shape-dependent integer: coordinates contribute two, constants zero, addition takes a maximum, multiplication takes a sum, and absolute value leaves the bound unchanged. It also counts every absolute-value node, without simplifying special constant values. Erasing all constant values preserves both quantities, even when equal constants at different positions are subsequently changed independently.

12.1 One precision for the entire vector field

For a finite family E_i, a clock coordinate θ, and rational $0 < \varepsilon \leq 1$, define the rational polynomials

$$V = 1 + \sum_i V_{E_i}, \qquad L = \varepsilon^{-1}(1 + \theta^2)V.$$

The module `FormulaPrecision.lean` proves $V \geq 1$, $L \geq 1$, and

$$|\widehat{E}_{i,L(x)}(x) - E_i(x)| \leq \frac{V_{E_i}(x)}{L(x)} \leq \frac{\varepsilon}{1 + \theta(x)^2}.$$

If D is the largest of the shape-dependent majorant degree bounds, then $\deg V \leq D$ and $\deg L \leq D+2$. Changing any rational constant, including the precision parameter, does not change this upper bound.

The module `FormulaSmoothness.lean` proves global smoothness of the smoothed family. At each absolute-value node, the radicand is $t(x)^2 + L(x)^{-2} > 0$, because $L(x) \neq 0$. Smoothness of square root away from zero, composition, and structural induction finish the proof. The reusable composition theorem permits any smooth nonvanishing precision, not just the polynomial chosen above.

The actual vector difference has sup norm at most the same clock tail; there is no dimension factor for this norm. Along a continuous supplied curve whose clock has derivative one,

$$\int_a^b \frac{\varepsilon}{1 + (c+t)^2}\, dt = \varepsilon\big(\arctan(c+b) - \arctan(c+a)\big) < \varepsilon\pi.$$

Here c is determined by the initial clock value and may be negative. The module `SmoothingPerturbation.lean` applies this comparison to the norm of the actual smoothing error on every finite forward segment. It does not require an ODE in the other coordinates. A forward-complete clock curve also has an integrable smoothing perturbation with total norm integral at most $\varepsilon\pi$; the passage to the ray first proves integrability from the uniform finite-interval bounds.

The polynomials and resource bounds are explicit recursive mathematical objects. The raw-code modules prove primitive recursiveness for the fixed formula templates and their input-dependent rational constants. The following compiler identifies the radical count with the length of its actual extracted program.

13 Auxiliary radicals and the quadratic penalty

The module `Simulation/PowerFraction.lean` handles the local algebra of radical extraction. Represent each rational expression by a polynomial numerator p and a denominator power L^k. Addition uses the common denominator $L^{k+\ell}$, and multiplication adds the exponents; neither operation cancels factors based on coefficient values. For a smoothing node, set

$$D = p^2 L^2 + L^{2k}.$$

If $L \geq 1$, then $D \geq 1$, including the case $k = 0$. For $L > 0$,

$$\sqrt{(p/L^k)^2 + L^{-2}} = \frac{\sqrt{D}}{L^{k+1}}.$$

This exact identity introduces no further approximation. The positivity of L is necessary to choose the displayed square-root branch.

13.1 An ordered compiler and its finite coordinates

`RadicalProgram.lean` represents a program by its ordered list of polynomial radicands. A base variable is tagged separately from a root index. Triangularity means that the radicand producing root j uses only roots with indices below j. Starting with arbitrary root values, the evaluator successively overwrites root j by the nonnegative square root of its radicand. Later writes cannot change an earlier radicand's value. Induction therefore proves every root equation in the *final* assignment, not merely at the moment that root was computed.

`FormulaRadicals.lean` compiles a formula from the inside out. Addition and multiplication concatenate their two child programs; an absolute-value node appends the radicand D above and returns its fresh root divided by L^{k+1}. The correctness theorem proves that actual sequential evaluation followed by evaluation of the returned fraction equals the smoothed formula. No root

equations are assumed in that theorem. The number of roots and the denominator exponent both equal the formula's syntactic absolute-value count, even when special coefficient values would permit cancellation.

`FormulaRadicalFamily.lean` applies the same construction to a finite list of formulas, allocating consecutive disjoint root ranges. One execution then represents all vector coordinates and any output formula included in the list. Every emitted radicand is at least one at *every* extended real state when $L \geq 1$, not only on the canonical root graph.

For completeness, `FormulaRadicalResources.lean` gives an explicit common degree bound. If d bounds the degree of L and $c(E)$ counts absolute-value nodes, define

$$\begin{aligned} B(x_i) &= 1, & B(a) &= 0, \\ B(E+F) &= B(E) + B(F) + d(c(E) + c(F)), & B(EF) &= B(E) + B(F), \\ B(|E|) &= 1 + 2B(E) + 2d(c(E) + 1). \end{aligned}$$

Induction bounds the output numerator and every emitted radicand by $B(E)$. Taking the maximum gives a family bound. Erasing or changing rational constant values leaves this bound unchanged.

Natural indices simplify concatenation, but the final polynomial system needs finitely many coordinates. `RadicalFiniteCoordinates.lean` explicitly substitutes the corresponding finite variable for each index below the root count and zero for any index outside it. The dependency bound proves that no out-of-range variable occurs. Renaming back therefore recovers each polynomial exactly, preserving its evaluation and total degree. This construction also handles a formula family with no radicals.

`FiniteRadicalFamily.lean` packages the resulting finite polynomial tuple and defines its canonical root graph by the actual sequential evaluator. Its equations, output values, and degree bounds follow from the preceding compiler proofs. `RadicalSmoothness.lean` proves smoothness of this graph by induction over the root program: each coordinate is a positive square root of a polynomial in already smooth coordinates. Polynomial evaluation uses only finitely many coordinates, so the intermediate natural-index representation causes no analytic infinite-dimensional assumption. Taking one more positive square root gives smooth fourth-root coordinates w_j with $w_j^2 = \xi_j$.

`RadicalLiftPolynomials.lean` makes the substitution $\xi_j \mapsto w_j^2$ in the actual rational polynomials. The resulting radicands $\mathcal{D}_j(x, w)$ are globally at least one, and the constructed canonical graph satisfies $w_j^4 = \mathcal{D}_j(x, w)$. `PolynomialCurveDerivative.lean` proves the polynomial chain rule on arbitrary finite coordinate types, so tagged base and root coordinates need no chosen enumeration.

13.2 Constructing the rational vector field

`PolynomialFraction.lean` keeps each fraction as an explicit pair of rational polynomials. Addition uses the product denominator, without cancellation; multiplication and finite sums are equally explicit. This makes every future denominator-clearing factor visible. `RadicalDependencies.lean` proves that $\mathcal{D}_j$ depends only on base variables and auxiliaries with indices less than j. In particular, its current and later formal partial derivatives vanish.

Let F be the constructed base fractions, and suppose velocities for the earlier auxiliaries have been constructed. Define

$$V_j = \frac{w_j}{4\mathcal{D}_j} \left(\sum_i \partial_{x_i} \mathcal{D}_j \, F_i + \sum_{h<j} \partial_{w_h} \mathcal{D}_j \, V_h \right).$$

`TriangularLift.lean` performs this finite recursion on polynomial fractions. Earlier coordinates are unchanged by later stages. Therefore the displayed numerator is the full directional derivative

of $\mathcal{D}_j$ along the final vector field: the omitted partials are zero. The global bounds $L \geq 1$ and $\mathcal{D}_j \geq 1$ imply that every constructed denominator is at least one, including far from the graph. The resulting rational field is consequently smooth everywhere.

`CompiledFormulaLift.lean` instantiates these facts with the compiler's actual radicands and output fractions. Each base coordinate selects its original formula-list position. On the generated graph its velocity is exactly that formula's smoothed value. No triangularity, denominator, or simulation assertion about the compiled system is left as an independent hypothesis; only the precision lower bound is needed.

`TriangularLiftInvariant.lean` now applies the full polynomial chain rule to an actual solution segment. It proves conservation of $w_j^4/\mathcal{D}_j$ and hence preservation of every exact quartic constraint. This is stronger than merely assuming a differential equation for each radicand in a scalar lemma.

Constraint preservation alone would not construct a lifted trajectory. `CompiledGraphTrajectory.lean` supplies that missing direction. Given a trajectory $x(t)$ of the smoothed base field, set $y(t) = (x(t), w_*(x(t)))$, where w_* is the actual positive root evaluator. This curve is differentiable. Differentiating its exact identities $w_j^4 = \mathcal{D}_j$ determines its auxiliary velocities in order. Indeed, after the earlier velocities are identified, the differentiated equation is

$$4w_j^3 \dot{w}_j = \dot{\mathcal{D}}_j.$$

Multiplying by w_j and using $w_j^4 = \mathcal{D}_j > 0$ gives exactly the constructed fraction V_j. This triangular tangent-uniqueness argument proves that y solves the actual rational lifted ODE. It does not assume a lifted solution or claim global existence of the base solution.

`ClockedMonitorLift.lean` makes this construction for the monitor's actual field and output formulas, with their shared precision polynomial. One fixed finite ordering allocates all roots. The output fraction on the graph equals the actual smoothed output, and a smoothed monitor trajectory lifts to the actual rational field. `ClockedMonitorResources.lean` proves that the exact root count and the precision and finite-radicand degree bounds do not change with the loaded input or precision coefficient. The final penalized homogeneous degree bound also uses the resource estimates below. The resource modules now also follow the actual recursive rational velocities. If all currently stored numerator and denominator degrees are at most b, the coordinate count is N, and the radicand degrees are at most d, one root update has the conservative degree bound $(N+1)(d+b)+1$. Iterating this bound for the fixed number of roots controls the actual stored fractions, without cancellation or dependence on the loaded input. `ClockedMonitorClearing.lean` applies the resulting bounds to joint field/output denominator clearing.

`PositiveDenominatorClearing.lean` performs the subsequent algebraic operation explicitly. For fractions p_i/q_i, put

$$Q = \prod_i q_i, \qquad P_i = p_i \prod_{j \neq i} q_j.$$

Then $Q \geq 1$ when each $q_i \geq 1$, and $P_i = Q(p_i/q_i)$ at every state. The degree bounds are the corresponding sums of numerator and denominator degrees. Field coordinates and output can share this multiplier. The module proves these polynomial identities. The finite-curve time-change and output-integral equivalence are proved separately below.

`LiftedMonitorExistence.lean` combines smoothed forward completeness with graph transfer. An affine test curve identifies the derivative of the graph map on the smoothed base velocity; the chain rule then gives the transfer on arbitrary time domains, including the one-sided derivative at zero. Consequently every canonical graph point starts an actual global forward trajectory of the rational lift. Neither off-graph completeness nor completeness after polynomial time acceleration is asserted.

13.3 Invariants and control of off-graph errors

The scalar identity behind the radical lift is proved in `Simulation/RadicalInvariant.lean`. Suppose $D > 0$ and a supplied solution segment satisfies

$$D' = \Lambda, \qquad w' = \frac{w\Lambda}{4D}.$$

Differentiating gives

$$\left(\frac{w^4}{D}\right)' = \frac{4w^3w'D - w^4D'}{D^2} = 0.$$

Consequently w^4/D is constant. An exact initial constraint $w^4 = D$ persists, and $w^2 = \sqrt{D}$ because $w^2 \geq 0$. This avoids a spurious negative square-root branch without imposing a sign condition on w. The proof uses no division by w, so it also covers $w = 0$ for inexact initial constraints. It does not assert that those inexact constraints decay.

When $D \geq 1$, factorization also gives the global estimate

$$|w^2 - \sqrt{D}| \leq |w^4 - D|,$$

because the other factor $w^2 + \sqrt{D}$ is at least one. For $D, E \geq 1$, the identity $D - E = (\sqrt{D} - \sqrt{E})(\sqrt{D} + \sqrt{E})$ gives $2|\sqrt{D} - \sqrt{E}| \leq |D - E|$. These are the elementary defect comparisons used by the following triangular construction.

13.4 Constructing the global defect weight

Write $\rho_j(x)$ for the canonical positive root of the original radicand $D_j(x, \rho_{<j})$. These are the square roots represented by w_j^2, not the fourth-root coordinates w_j themselves. We first need polynomial bounds for the canonical roots. For a rational polynomial p, let $m(p)$ be the sum of the absolute values of its coefficients. On a coordinate cube of radius $R \geq 1$,

$$|p(z)| \leq m(p)R^{\deg p}.$$

Suppose polynomial majorants $S_i(x) \geq |\rho_i(x)|$ have already been constructed for $i < j$. Use

$$R_j(x) = 1 + \sum_a x_a^2 + \sum_{i<j} S_i(x)^2, \qquad S_j(x) = 1 + m(D_j)R_j(x)^{\deg D_j}.$$

The radius dominates every base coordinate and every earlier root. Later roots can be set to zero because D_j does not depend on them. The inequality $\sqrt{t} \leq 1 + |t|$ proves the next majorant. `RadicalMajorant.lean` implements this finite recursion literally, including the empty system.

Now fix any auxiliary assignment w, however far from the graph. Both w_i^2 and $\rho_i(x)$ lie in the interval bounded by $w_i^2 + S_i(x)$. A single polynomial radius $R(x, w) \geq 1$ therefore contains both extended points (x, w^2) and $(x, \rho(x))$. Applying the scalar mean-value theorem along their joining segment gives

$$|p(x, w^2) - p(x, \rho)| \leq \sum_i C_{p,i}(x, w)|w_i^2 - \rho_i|, \qquad C_{p,i} = m(\partial_{\rho_i} p)R^{\deg p}.$$

These weights are nonnegative rational polynomials. If a root does not occur in p, its formal partial derivative is zero and its weight is identically zero. The formal proof uses coordinatewise partials rather than an unspecified Lipschitz constant; this retains triangularity. The modules are `PolynomialDifferenceBounds.lean` and `RadicalComparison.lean`.

Define the actual quartic defects and their norm by

$$r_j = w_j^4 - D_j(x, w^2), \qquad E = \left(\sum_j r_j^2 \right)^{1/2}.$$

The two elementary square-root inequalities above and the polynomial comparison yield, with $d_j = |w_j^2 - \rho_j(x)|$,

$$d_j \le |r_j| + \sum_{i<j} C_{D_j,i} d_i.$$

We have discarded a factor $1/2$ to simplify the bound. Define actual polynomials recursively by

$$W_j = 1 + \sum_{i<j} C_{D_j,i} W_i.$$

Since $|r_j| \le E$, induction gives $d_j \le W_j E$. Thus any numerator has error at most $(\sum_i C_{p,i} W_i)E$. `TriangularErrorWeights.lean` isolates the reusable substitution argument, and `RadicalDefectBounds.lean` supplies its actual coefficients and defects. There is no smallness assumption on E.

Each compiled output divides its numerator by a common precision power $L^e \ge 1$. This cannot enlarge the error. Sum the finitely many output weights and add 1 to obtain one polynomial $W \ge 1$ controlling every field coordinate and the output. `CompiledFormulaError.lean` instantiates all root equations and positivity hypotheses with the compiler's proved properties. The associated resource modules bound these actual weights' degrees using only the original degree bounds and root count, not the coefficients.

`CompiledPolynomialFormula.lean` proves a useful exactness fact: an abs-free formula allocates no roots, so its extracted value is correct for every auxiliary assignment, even inside a family containing other formulas with radicals. Therefore the lifted monitor still satisfies $\theta' = 1$, $u' = -v$, and $v' = u$ off graph. The buffer and output errors are bounded by the constructed WE.

Finally, `ClockedMonitorPenalty.lean` constructs the polynomial

$$P = (1 + \theta^2) W^2 \sum_j r_j^2 = (1 + \theta^2)(WE)^2$$

and subtracts it from the actual rational output, keeping its denominator unchanged. The penalty is nonnegative everywhere and zero on the constructed graph. Consequently the penalized output there is exactly the smoothed monitor output. This is a pointwise algebraic theorem; the following integral argument supplies its dynamical meaning.

The module `Simulation/QuadraticPenalty.lean` proves the separate numerical estimate which later absorbs the weighted defect cost. For every $a > 0$ and real c, R, completing the square gives

$$cR - aR^2 \le \frac{c^2}{4a}.$$

Along a unit-speed clock, use $a = 1 + \theta^2$ and integrate. For every continuous error function R on a supplied finite forward interval,

$$\int (cR - (1 + \theta^2) R^2)\, dt \le \frac{\pi c^2}{4}.$$

The bound does not depend on the size of R or the initial clock value.

13.5 The rational-monitor criterion

Set $R = WE$ and $\sigma = \epsilon/(1+\theta^2)$. Along any finite trajectory of the actual rational lift, its base projection satisfies the original monitor equation with a continuous buffer forcing of norm at most $\sigma + R$. This is an identity obtained by subtracting the two actual fields; the clock and oscillator components cancel exactly. The rational output differs from the original monitor output by at most $\sigma + R$. Consequently, if the source halts in T instructions and $C = 4\pi(T+3)$, the proved monitor estimate gives

$$\int o\,dt \le C\left(256\left(\epsilon\pi + \int R\,dt\right) + 2\right).$$

Subtract the actual penalty from the rational output. With $c = 256C + 1$, the preceding square-completion inequality yields the uniform bound

$$\int g\,dt \le 2C + c\epsilon\pi + \frac{\pi c^2}{4}.$$

It holds for every finite solution segment, regardless of initial state or length. The proof requires neither a small defect nor global existence away from the graph. The residual identity is in `ClockedMonitorLiftForcing.lean`. Its mass bound is proved in `ClockedMonitorLiftIntegral.lean`, and the square-completion argument is in `PenalizedMonitorIntegral.lean`.

Conversely, the actual global smoothed nonhalting witness lifts to the canonical rational graph. Its penalty is zero, and the total difference between original and smoothed output on any finite interval is at most $\epsilon\pi$. Its finite penalized-output integrals are therefore unbounded. `NonhaltingLiftedOutput.lean` proves this statement for the constructed trajectory.

Let $\mathcal{B}(F, g)$ mean that one real constant bounds $\int_0^T g(y(t))\,dt$ for every $T \ge 0$ and every continuous solution segment of $y' = F(y)$ on $[0, T]$. The endpoint `RationalMonitorCriterion.halts_iff_bounded_penalizedOutput` proves

$$\text{the source halts} \iff \mathcal{B}(\text{actual rational field}, \text{actual penalized output})$$

for $0 < \epsilon \le 1/1024$. All field, output, and simulation data are constructed, not supplied as hypotheses.

13.6 Clearing denominators on finite trajectories

The common denominator Q is a polynomial with rational coefficients and $Q \ge 1$ on the whole state space. Set

$$P = QF, \qquad b = Qg,$$

where F is the rational field and g its penalized output. Both P and b are polynomials. The penalty did not change the stored output denominator, so the same Q clears the field and the output.

Here it is important to compare finite trajectories, not to assume that the polynomial field is complete. Given a trajectory y of F on $[0, T]$, define

$$s(t) = \int_0^t \frac{du}{Q(y(u))}.$$

Continuity and positivity make this clock strictly increasing, with a continuously differentiable inverse on the interior of its finite image $[0, S]$. The reparametrized curve $z(s) = y(t(s))$ satisfies $z' = P(z)$, and change of variables gives

$$\int_0^S b(z(s))\,ds = \int_0^T g(y(t))\,dt.$$

Starting with a finite trajectory of P gives the reverse construction. Thus exactly the same constant bounds the output integrals for either system. Neither direction requires a bound on Q outside the compact image of the supplied trajectory.

The reusable result `hasBoundedFiniteOutput_iff_timeScaled` proves this equivalence for continuous positive speeds on a real normed space. Its finite-clock construction extends the speed by positive constant tails, obtains a global inverse clock, and then restricts to the required finite interval. This device handles the endpoints without extending the trajectory as an ODE solution. Zero-length intervals are included.

Consequently, `ClockedMonitor.halts_iff_bounded_polynomialOutput` proves

$$\text{the source halts} \quad \Longleftrightarrow \quad \mathcal{B}(P, b)$$

for the actual constructed polynomial pair. Its degree bounds depend only on the fixed register program, not on the loaded input.

13.7 From bounded output to homogeneous stability

Let P be a polynomial vector field and b a polynomial scalar output on $\mathbb{R}^n$. Choose an odd integer $\ell \geq 1$ such that every term of P has degree less than ℓ and every term of b has degree less than $\ell - 1$. For $Y = (y, z) \in \mathbb{R}^{n+1}$, homogenization defines polynomials

$$A(Y) = (z^\ell P(y/z), 0), \qquad \beta(Y) = z^{\ell-1} b(y/z).$$

These expressions initially describe the finite chart $z \neq 0$; the polynomial definitions make sense at $z = 0$ as well. Set

$$F(Y) = |Y|^2 A(Y) - (Y \cdot A(Y))Y + |Y|^2 \beta(Y) Y.$$

This is homogeneous of odd degree $\ell + 2$. The strict degree bounds imply $A = 0$ and $\beta = 0$ on the equator $z = 0$.

On the Euclidean unit sphere, the angular vector field and radial reward of F are

$$A(Y) - (Y \cdot A(Y))Y, \qquad \beta(Y),$$

respectively. The preserved angular stability theorem says that the origin is Lyapunov stable, with existence, exactly when one constant bounds all forward integrals of this reward along sphere trajectories. The coordinate-space formulation uses the algebraic sum of squares for the sphere; it does not replace that sphere by the sup-norm unit sphere.

Every equator point is stationary and contributes zero reward. An angular trajectory starting outside the equator cannot reach it in finite time: backward uniqueness would identify the trajectory with the stationary equator solution. Thus the quotient $x = y/z$ is defined on every finite segment of such a trajectory. Differentiation gives

$$x' = z^{\ell-1} P(x), \qquad \beta(Y) = z^{\ell-1} b(x).$$

Since $\ell - 1$ is even, the common speed is positive on either hemisphere. Finite time change therefore transfers any uniform source-output bound to the angular reward integrals.

For the converse, start with any finite trajectory of $x' = P(x)$ and form its positive central lift

$$c(x) = (1 + |x|^2)^{-1/2}, \qquad Y = (c(x)x, c(x)).$$

The lift lies on the unit sphere. Its velocity is the angular field multiplied by $c^{-(\ell-1)}$, and its radial reward multiplied by the same factor is exactly $b(x)$. Removing this positive speed gives a finite

angular trajectory with the same output integral. Finite-interval uniqueness identifies it with the canonical angular flow used in the stability criterion. This proves

$$0 \text{ is Lyapunov stable for } F \quad \Longleftrightarrow \quad \mathcal{B}(P, b).$$

Neither implication assumes global completeness of P. The module and theorem are

`ProjectiveStabilityCriterion.lean`,
`lyapunovStable_fieldCode_iff_boundedFiniteOutput`.

The projective field itself is compiled by primitive-recursive sparse-code operations. The stated degree conditions concern raw retained terms, not just decoded degree: canceled high-degree terms must first be removed. `PolynomialCodeTrim.lean` proves an effective filter that preserves the decoded polynomial and converts its semantic degree bound into the required raw-term bound. If both decoded degrees are at most d, the module `PolynomialOutputReduction.lean` chooses $\ell = 2d+3$ after trimming. It produces a homogeneous field of degree $2d+5$ in $n+1$ variables, primitive-recursively from the original n-variable field and output. The strict inequalities at the equator follow from this choice.

13.8 Compiling the actual register monitor

The module `ClockedMonitorPolynomialCode.lean` now constructs that field/output pair from the loaded register values. Its correctness proof identifies both decoded polynomials with the semantic monitor, not merely with polynomials having the same degree bounds. The compiler first instantiates fixed formula trees, extracts the ordered radicals, and forms the literal rational fractions for the base and auxiliary velocities. It computes the actual polynomial comparison weights and quartic-defect penalty, clears the field and output denominators together, and subtracts the same clearing factor times the penalty from the output. Each operation is primitive recursive on sparse rational codes.

Only the rational coefficients depend on the loaded input. The coordinate enumeration, number of roots, formula order, and degree bound d are fixed by the program. Equality casts identify the input-dependent semantic root type with this fixed code space; they do not choose a new runtime enumeration. Coefficient mass and degree are computed after coefficient cancellation, so redundant sparse presentations do not invalidate the bounds or the strict inequalities used in homogenization.

Fix the smoothing accuracy to $\varepsilon = 1/1024$. Composing the exact decoder, the polynomial-monitor criterion, finite-coordinate conjugacy, and the projective theorem gives

$$\text{the fixed program halts on the loaded input} \quad \Longleftrightarrow \quad \mathrm{HLS}_{\mathbb{Q}}(n+1, 2d+5)(F_{\mathrm{input}}).$$

The module `HomogeneousStabilityReduction.lean` proves this equivalence, homogeneity of every output code, and primitive recursiveness of the map. The theorem has no supplied simulation, source-hardness, or effectivity hypothesis. Its stability notion is exactly the preserved `HGAS.HLSQ`, including forward existence near the origin.

13.9 Unconditional cubic and cofinal HLS hardness

Apply this compiler to the single five-register program derived earlier from Mathlib's universal evaluator. The initialization puts the input in register zero and sets the other registers to zero. This yields fixed integers $n_0 \geq 2$ and $k_0 \geq 1$, and a primitive-recursive raw polynomial map f, such that

$$\mathrm{UniversalHalts}(c, a) \quad \Longleftrightarrow \quad \mathrm{HLS}_{\mathbb{Q}}(n_0, 2k_0+1)(f(c, a)).$$

Both integers are closed symbolic terms independent of (c, a); the development does not evaluate them to numerical literals.

The preserved cubicization theorem is a computable many-one reduction from this HLS language to degree three in the fixed dimension N_0, given by `cubicLiftCodeDimension` at (n_0, k_0). Its source-dimension hypothesis is discharged because the monitor has a clock coordinate and homogenization adds another coordinate. Dimension padding and odd-degree raising then give every $N \geq N_0$ and every odd $d \geq 3$.

The final module, `UniversalStabilityReduction.lean`, proves that each resulting language is many-one hard for recursively enumerable sets, is not computable, and has a complement that is not recursively enumerable. The formal statements have no source-hardness assumption. They do not assert that HLS itself is recursively enumerable, nor do they settle its low-dimensional cases. The earlier conditional transfer theorems remain unchanged and are used in this composition.

14 Validation and limits

The source-level ordinary-time equivalence, explicit Fourier and primitive bounds, printed-gain compact graph measure, and two-sided reward gap are proved. The halting-to-HLS reduction is also complete, from Mathlib's universal evaluator through the actual rational and polynomial monitor codes to the homogeneous field. Its final hardness theorems have no source-hardness, simulation, or compiler-effectivity premise.

Independent audits checked the statements and the main semantic connections: finite-time output estimates, arbitrary off-graph states, positive denominator clearing, finite-curve projective charts, code realization, graph invariance under the reciprocal clock, and both sides of the compact reward gap. Regression examples include a one-step halting program, an infinite increment loop, canceled polynomial terms, zero-dimensional Fourier and chart cases, and a noninjective compact graph parametrization.

The command `lake build Stability` builds the additions. Two scripts provide separate checks:

- `scripts/StabilityRegression.lean` checks the examples;
- `scripts/StabilityAxiomAudit.lean` reports axiom dependencies.

The policy remains: no project axioms, placeholders, or unsafe proof shortcuts. Standard dependencies on `propext`, `Classical.choice`, and `Quot.sound` are permitted and reported. The published build and import-closure audit remain separate and continue to check the unchanged 837-module source tree. The ledger records the validation commands and current counts.

These results do not determine the three-dimensional cubic problems, evaluate the symbolic fixed dimensions, or prove recursive enumerability of HLS or general polynomial GAS/LAS. They also do not claim that every printed intermediate statement is exported verbatim: the ledger distinguishes exact statements from equivalent formulations and lists results outside this formalization's scope.

References

[1] I. O. Shevchenko, J. Liu, and X. Liu, *Global asymptotic stability of homogeneous polynomial vector fields is undecidable*, arXiv:2610.12434. https://arxiv.org/abs/2610.12434. For the intermediate statements discussed here, the precise working manuscript is Overleaf commit `6f5bced`, especially `sec08_fourier_to_orbits.tex`, `sec09_orbits_to_average.tex`, and `sec11_proof_of_main.tex`.

[2] M. Korda, *Lyapunov stability of polynomial vector fields is undecidable*, arXiv:2609.22058v2. https://arxiv.org/html/2609.22058v2. The relevant statements are Definition 1.1 and Lemma 1.5 (stability with existence and homogeneous linear bounds), Sections 3–4 (restarts and the robust monitor), Lemma 4.1 (cubic relaxation), Proposition 4.5 (the all-initial-state monitor estimate), and Sections 5–6 (algebraization and homogeneous lifting).